Lab 1: Parity-Based Selection on LC-3

September 27, 2026

1 Overview

This lab familiarizes you with the LC-3 instruction set—particularly logical and branching instructions—by implementing a simple parity-based selection in machine code only. Your program should accept one integer input and two candidate values and, based on whether the integer is even or odd, produce one of the two candidate values as the result. You must encode your program as a sequence of 16-bit binary instructions that can be executed by an LC-3 simulator.

2 Problem Description

Given a 16-bit two's-complement integer `key` and two 16-bit values `evenVal` and `oddVal`, your program should compute

$$\text{result} = \begin{cases} \texttt{evenVal}, & \text{if key is even,} \\ \texttt{oddVal}, & \text{if key is odd.} \end{cases}$$

Recall that, in two's-complement representation, an integer is even exactly when its least significant bit is 0, and odd exactly when its least significant bit is 1.

Your task is to write an LC-3 program that implements this selection, where

- `key` is an arbitrary 16-bit two's-complement integer and is initially in register R0.
- `evenVal` is an arbitrary 16-bit two's-complement integer in R1.
- `oddVal` is in R2.
- The program should store the result in R0.

2.1 Examples

key (R0)	evenVal (R1)	oddVal (R2)	Expected result (R0)
0	5	9	5
7	5	9	9
−4	−3	4	−3
−7	−7	−2	−2

2.2 Requirements and Tips

- The program must start at address x3000 (binary `0011 0000 0000 0000`).
- Use the HALT trap service routine to terminate your program (TRAP x25, binary `1111 0000 0010 0101`).
- You may use any LC-3 instructions as needed.

- Inputs are already placed in the specified registers when your program starts—do not read from memory or the keyboard.
- Submit a plain-text listing with one 16-bit binary word per line.
- Include brief inline comments explaining each instruction's purpose. For example,
  `0101 011 000 1 00001 ; R3 = R0 & 1.`

3 Reference Approaches

Below are two possible approaches to implement the parity-based selection. You may use either of them or devise your own solution.

3.1 Approach A — Branching Solution

Basic Idea

This approach treats the problem as a small if-else statement. First isolate the least significant bit of `key`, then branch according to whether that bit is zero:

1. Compute `key & 1` to determine the parity and set the condition codes.
2. If the result is 0, branch to the even block that copies R1 (`evenVal`) to R0.
3. Otherwise, fall through and copy R2 (`oddVal`) to R0, then use an unconditional branch to skip the even block.
4. Finally, halt the program.

Tips

- The LC-3 `AND` instruction can be used with the immediate value 1 to inspect the least significant bit.
- The `AND` result automatically updates the condition codes, so a zero branch can be used immediately afterward.
- Use an unconditional branch (BRnzp) to jump around the other selection block.
- Keep labels straight on your scratch paper even though you submit binary only.
- Review how to encode PC-relative branch targets; this is often the most error-prone part.

3.2 Approach B — Branchless Bit-Masking Solution

Basic Idea

This approach first computes the parity bit

$$p = \texttt{key} \ \& \ 1,$$

then turns it into an all-ones or all-zeros mask using two's complement:

$$\text{mask} = -p = \begin{cases} \texttt{0xFFFF}, & \text{if key is odd,} \\ \texttt{0x0000}, & \text{if key is even.} \end{cases}$$

Then select the desired value via

$$\text{result} = (\texttt{oddVal} \ \& \ \text{mask}) \mid (\texttt{evenVal} \ \& \ \sim \text{mask}).$$

Tips

- Use `AND` with immediate 1 to obtain the parity bit.
- Use `NOT` and `ADD` to compute the two's complement and obtain the mask.
- Since there is no bitwise OR instruction in LC-3, you may use De Morgan's law:

$$A \vee B = \neg(\neg A \wedge \neg B).$$

  However, in this particular case, the two masked values never have a 1 in the same bit position, so the OR operation can be replaced by an arithmetic operation supported by LC-3. Think carefully.
- Remember that `0xFFFF` is the 16-bit two's-complement representation of -1.

4 Testing and Debugging

You can use LC3Tools to run and debug your program. Refer to the official Installation Guide and User Guide to learn how to use this tool.

5 Submission

Organize your submission as follows:

```
PB********_Name_lab1.zip
|-- PB********_Name_report.pdf
`-- lab1.bin.txt
```

Your report should include:

- Your name and student ID.
- A brief description of your solution.
- A few test cases you used to verify your program. Include initial values in registers R0, R1, and R2 as well as the final value in R0.
- (Optional) Any challenges you encountered and how you addressed them.